

NeuroGraph

Abdulrahman Alkhateeb

Motivation

- LLMs are fluent but hallucinate and lack structured grounding
- Knowledge graphs are precise but rigid and expensive to build manually
- LLMs can build KGs; KGs can constrain LLMs
- Bidirectional integration captures the strengths of both

Two Paradigms

- LLM-augmented KG: an LLM reads unstructured text and produces a structured graph
- KG-enhanced LLM: a KG supplies verified context that grounds the LLM's answers
- This project implements both, over a single shared graph

What Was Built

- Gradio web app with two tabs and a shared, per-session knowledge graph
- Interactive network visualization (drag, scroll-zoom, hover tooltips, click-to-highlight)
- Backed by a fully local LLM via Ollama — no external API calls

Application 1: LLM-augmented KG

- **Input:** raw text (article, notes, transcript)
- **Pipeline:**
 - LLM extracts entities (id, type, description) and relationships (source, relation, target)
 - Validation layer normalizes types, dedupes, drops dangling references
 - Merges into the in-memory store, reporting added vs. updated counts
- **Output:** entities table, relationships table, interactive graph

Application 2: KG-enhanced LLM

- **Input:** natural-language question
- **Pipeline:**
 - Graph serialized to plain text (truncated at line boundary if oversized)
 - System prompt restricts the LLM to facts present in the graph
 - Falls back to "the graph does not contain enough information" if grounded answer impossible
- **Output:** grounded answer + the exact context used (transparency panel)

Architecture

- `config.py` — env-driven shared config
- `kg_store.py` — `KnowledgeGraphStore` wraps `networkx`, returns `IngestionResult` diffs
- `llm_augmented_kg.py` — extraction app + JSON validation
- `kg_enhanced_llm.py` — QA app
- `kg_visualizer.py` — `pyvis` → self-contained interactive HTML
- `app.py` — Gradio UI, per-session `gr.State`

Tech Stack

- Python 3.10+
- Gradio — UI framework
- Ollama + qwen2.5:3b — local LLM runtime
- LangChain — prompt templates and chain plumbing
- NetworkX — directed graph data structure
- pyvis / vis.js — interactive network visualization

Next Steps

- Possible extensions:
 - Larger / instruction-tuned model for richer extraction
 - Persistent storage (Neo4j, SQLite) instead of in-memory
 - Subgraph-aware retrieval for QA on large graphs
 - Multi-hop reasoning over the graph structure

Thank you!